Speed and Energy Aware Scheduling for LLM Agent Requests

EE538 Phase I Proposal Track B LLM and MLSys Module

1 People and Repository

Cris Hu jinhaoh@usc.edu Zheyu Yang zheyuyan@usc.edu Alicia Ma xiaodanm@usc.edu

Team GitHub repository: https://github.com/Cris109486/EE538_project

2 Problem and Interface

We will implement a C++17 discrete-event inference scheduler that explores completion time, throughput, and energy for agent-originated LLM calls. Team-written deterministic fixtures and a seeded workload generator produce JSON inputs specifying arrivals, prompt lengths, output limits, request dependencies, cancellations, KV capacity, per-iteration token limits, workflow deadlines, and an execution cost profile. The driver keeps realized output lengths private; policies see only limits and estimates. Outputs include batch/event logs and CSV metrics for makespan, mean/P95 request latency, time to first token (TTFT), inter-token gap, request throughput, output-token throughput, total/component energy, deadline success, peak KV use, and measured scheduler runtime/memory. Request throughput is completed requests divided by makespan; output-token throughput is generated output tokens divided by makespan. Example fixture: a request has 4 prompt tokens and 2 output tokens. With a prefill chunk of 2, two prefill steps each cost 1 ms/0.2 J; the final prefill produces the first output token, and one 1 ms/0.1 J decode step produces the second. The request therefore completes in 3 ms using 0.5 J, excluding background energy. These values are test assumptions, not hardware measurements.

3 Track and Proposed Scope

This project is a Track B LLM and MLSys module. A driver submits requests over simulated time, while the C++ core handles admission, partial prefill, one-token decode steps, completion, cancellation, dependency release, and batch selection. KV capacity is conservatively reserved from prompt length plus maximum output length and released at completion or cancellation. Arrivals and cancellations during a batch take effect at the next batch boundary.

Speed priority seeks a short workflow makespan. Energy priority seeks low total joules subject to a configured workflow deadline and the same resource constraints; varying the deadline exposes the trade-off. At each decision point, bounded candidates vary prefill chunk sizes and request composition. Speed scoring favors predicted normalized progress per unit time, while energy scoring favors progress per joule among candidates that pass an estimated deadline-feasibility check. Deterministic ties and waiting-time aging prevent indefinite deferral. Both policies are heuristics, not global optima.

The prototype covers one request lifecycle, fixed costs, and fixed-chunk first-come, first-served (FCFS). The complete Phase II system adds interacting lifecycles, custom data structures, dependency management, bounded candidate construction, both objectives, model-sensitivity analysis, independent checking, and reproducible experiments. Non-goals are real LLM/agent execution, GPU kernels, physical KV buffers or paging, hardware power control, and a production server. Stretch goals are measured profile calibration and a Power project interface; no essential functionality is deferred to stretch work.

4 Course Mapping and Algorithmic Basis

Custom components include BinaryHeap<T, Compare> for events, an adjacency-list TaskGraph, and RingBuffer<T> for recent prediction-error statistics. Algorithms include Kahn topological validation and resource-constrained greedy admission/batch construction. Classes include Simulator, KVResourceTracker, BatchBuilder, CostModel, EnergyLedger, and MetricsCollector; ISchedulingPolicy uses virtual dispatch for FCFS, speed-priority, and energy-priority policies. RAII, value ownership, stable IDs, encapsulation, and const-correctness define lifetimes. STL containers support storage and iteration; JSON parsing, CMake, and Python plotting are reused infrastructure. Core execution and policy logic are ours.

Prior systems motivate individual mechanisms. Orca [1] motivates iteration-level scheduling; Sarathi-Serve [2] motivates chunked prefill and mixed prefill/decode batches; vLLM/PagedAttention [3] motivates explicit KV-capacity accounting for variable-length requests; Vidur [4] motivates separating workloads, execution profiles, simulation state, and metrics; and Stojkovic et al. [5] motivate evaluating energy with latency and throughput under performance constraints. We do not reproduce these systems or their hardware results. Our custom contribution is a deterministic C++ simulator with dependency-aware lifecycles, conservative KV reservation, bounded candidate construction, and two scoring policies.

5 Correctness and Independent Reference

We use hand-calculated end-to-end fixtures and an independent Python checker for batch token counts, request progress, dependencies, KV reservations/releases, timing intervals, energy, and summary metrics. Tests cover heap order, ring-buffer wrap, graph endpoints, exact capacity, mixed phases, tied events, mid-batch arrivals/cancellations, resource release, starvation, and termination. The program rejects cycles, invalid IDs, impossible limits, missing profiles, and negative or non-finite quantities. All policies share the same deadline and accounting semantics. CTest covers normal, boundary, and error cases; sanitizers are used where supported.

6 Evaluation and Model Credibility

Research question: when do chunking and cost-aware batch selection improve the speed-energy trade-off under KV pressure? We compare fixed-chunk FCFS, speed priority, and energy priority on identical work. A seeded generator creates independent requests, chains, branches, and fork-join workflows at scales of 10, 100, and 1,000 requests. Experiments vary prompt/output mix, arrival burstiness, dependency shape, KV capacity, chunk size, deadline, and execution-profile shape.

We report makespan; mean/P95 latency; TTFT; inter-token gap; waiting time; request and output-token throughput; total/component energy; joules per request and per output token; deadline success; and peak KV use. Timelines, speed-energy points, and scaling plots summarize the results. Execution profiles map phase, batch tokens, active sequences, and context length to time and energy, with background energy accounted for separately. The initial documented synthetic profile must not bake in an advantage for any policy. We test ideal agreement, prediction error, multiple profile shapes, and cases where optimization loses. Without valid calibration data, conclusions remain within the stated model and make no measured hardware-energy claim.

We record seeds, machine, compiler/options, configuration versions, and raw data. Each setting uses at least 10 workload seeds, and C++ timing is repeated five times; we report the median and interquartile range. Measured C++ runtime and peak process memory are separated from simulated LLM time/energy and driver overhead. With N requests, E dependencies, Q pending events, R active requests, and K candidate batches, graph validation is O(N + E), event-heap operations are O(log Q), and bounded candidate construction/scoring is O(KR).

7 Resources and Fallback

Resources are a Windows CPU laptop, a documented C++17 compiler, CMake/CTest, Git/GitHub, Python 3, a JSON library, NumPy, and Matplotlib. The planning budget is 8 GB RAM and 2 GB storage. No GPU, paid service, model weights, commercial EDA tool, or restricted dataset is required. The repository provides build presets, sample inputs, fixed seeds, and reproduction commands while excluding local environments and build artifacts. If verified external calibration data are unavailable, the fallback is explicitly labeled synthetic profiles.

8 Milestones Responsibilities and Risk

Cris Hu owns the driver, state machine, TaskGraph/BinaryHeap, input validation, and integration. Zheyu Yang owns KV accounting, batch construction, and scheduling policies. Alicia Ma owns C++ execution profiles, history/energy/metrics, and independent Python checks/experiments. Each member tests their module, cross-reviews a teammate, and contributes to the report/demo. Week 1: contracts, reference checker, and a hand-checkable lifecycle. Week 2: integration and difficult state transitions. Week 3: controlled experiments and sensitivity analysis. Final days: clean-checkout reproduction, README evidence map, 4-6 page report, demo, and fixed Git tag.

The main risk is that an uncalibrated cost model drives the conclusions. We mitigate it with independent accounting, explicit assumptions, multiple profiles, prediction-error sensitivity, negative results, and limited claims. We will obtain instructor/TA approval before Phase II and before any substantial change to the topic, algorithmic scope, or required resources.

References

[1] G.-I. Yu et al., "Orca: A Distributed Serving System for Transformer-Based Generative Models," OSDI, 2022.

[2] A. Agrawal et al., "Taming Throughput-Latency Tradeoff in LLM Inference with Sarathi-Serve," OSDI, 2024.

[3] W. Kwon et al., "Efficient Memory Management for Large Language Model Serving with PagedAttention," SOSP, 2023.

[4] A. Agrawal et al., "Vidur: A Large-Scale Simulation Framework for LLM Inference," MLSys, 2024.

[5] J. Stojkovic et al., "Towards Greener LLMs: Bringing Energy-Efficiency to the Forefront of LLM Inference," arXiv:2403.20306, 2024.